

Verdant-Search: Bridging Vision and Text in Distributed Search Through Multimodal Learning-to-Rank

Student: CAI HONGYI, S2175463

Supervisor: CHIAM YIN KIA

Software Engineering Department, Universiti Malaya

March 26, 2026

Agenda

1. Fulfilment of Project Objectives
2. Fulfilment of Requirements
3. Design and Technical Implementation
4. System Complexity
5. Error Handling
6. Testing Techniques
7. Stakeholder Collaboration Initiative (FYP2)
8. Logbook (FYP2)

Project Objectives #1

Objective

To **identify** existing search engine architectures, learning-to-rank approaches, vision language models, and retrieval-augmented generation techniques through literature review.

Fulfilment Methods

- Literature Review conducted
 - 9 research areas surveyed (architecture, keyword search, vector embeddings, ANN, classical LTR, LLM ranking, VLMs, RAG systems, benchmark datasets)
 - System Archeology for existing applications
- Comparison with existing systems

Research Gaps Identified

1. **No multimodal reranking** — all existing LTR methods text-only
2. **Visual evidence underutilised** in RAG systems
3. **One-shot retrieval** — no iterative LLM-driven refinement
4. **VLM inference cost** — efficiency challenges at scale

Project Outcome

FR and NFR of Verdant-Search address all identified gaps. Evaluation results confirm this.

Project Objectives #2

Objective

To **design and develop** a multimodal search engine integrating distributed crawling, hybrid retrieval, multimodal listwise reranking, and iterative RAG with query rewriting.

Functional Requirements Fulfilled

User Management, Search & Retrieval, Data Ingestion, Analytics & Monitoring, RAG Generation Modules

Non-Functional Requirements Fulfilled

- Security — Authenticity
- Security — Confidentiality
- Security — Integrity
- Performance — Latency & Capacity
- Reliability — Fault Tolerance
- Maintainability — Modularity
- Usability — Learnability

Conclusion

All FR and NFR are fulfilled.

Project Objectives #3

Objective

To **evaluate** system effectiveness through ranking metrics, latency benchmarks, and answer quality assessments.

Fulfilment Methods

- Unit, Integration, System and Validation Testing
 - Ranking metric evaluation (NDCG, MRR)
 - Latency benchmarks (P90 under 4s target)
 - Answer quality assessment
- User Acceptance Test (UAT) conducted

Module / NFR	Score (1–5)
User Management Module	4.5
Search & Retrieval Module	4.8
Data Ingestion Module	4.6
RAG Generation Module	4.7
Overall Usability	4.7
Performance — Latency	4.5
UI Learnability	4.8

Conclusion

UAT result shows **high acceptance** of the system.

Fulfilment of Requirements

Functional Requirements

Fulfilment of Requirements

FR Module	Fulfilled
User Management Module	✓
Search & Retrieval Module	✓
Data Ingestion Module	✓
Analytics & Monitoring Module	✓
RAG Generation Module	✓

NFR	Fulfilled
Security — Authenticity	✓
Security — Confidentiality	✓
Security — Integrity	✓
Performance — Latency & Capacity	✓
Reliability — Fault Tolerance	✓
Maintainability — Modularity	✓
Usability — Learnability	✓

User Management Module

[PLACEHOLDER]

images/diagrams/uc.user_management.png

*User Management Module Use Case
Diagram (UC-01 to UC-11)*

[PLACEHOLDER]

images/screenshots/user/login-profile_screens.png

*Login page (email/SSO) and User Profile
settings page*

Module	ID	Requirement	Priority
User Mgmt	FR-1.1	User shall register an account with email and password.	High
	FR-1.2	System shall validate email format and password strength.	High
	FR-1.3	System shall send verification email upon registration.	Medium
	FR-1.4	User shall login with email/password or SSO authentication.	High
	FR-1.5	System shall support SSO providers (LDAP, Okta, Azure AD).	High
	FR-1.6	System shall authenticate user credentials securely.	High
	FR-1.7	User shall manage profile with configurable preferences.	Medium
	FR-1.10	User shall view and manage their chat history.	Medium
	FR-1.15	System shall implement session management with timeout.	Medium

Search & Retrieval Module (1/2) — Hybrid Search

[PLACEHOLDER]

images/diagrams/uc_search_retrieval.png

*Search & Retrieval Module Use Case
Diagram (UC-12 to UC-24)*

[PLACEHOLDER]

images/screenshots/search/main_search_interface.png

*Main Search Interface — query input,
BM25 + vector hybrid results panel*

Module	ID	Requirement	Priority
Search & Retrieval	FR-2.1	User shall submit search query through text input.	High
	FR-2.2	System shall sanitize and normalize query text.	High
	FR-2.4	System shall perform keyword search using BM25.	High
	FR-2.5	System shall build inverted index with term posting lists.	High
	FR-2.6	System shall perform vector search using HNSW.	High
	FR-2.7	System shall encode queries with Sentence-BERT embeddings.	High
	FR-2.8	System shall support hybrid search with RRF fusion.	High
	FR-2.9	System shall combine BM25 and vector scores with weighted fusion.	High
	FR-2.14	System shall apply multimodal reranking to top candidates.	High
	FR-2.15	System shall use BLIP-2 for cross-modal relevance scoring.	High

Search & Retrieval Module (2/2) — Conversational Search

[PLACEHOLDER]

`images/screenshots/search/results_conversational.png`

Search results with highlighted snippets and multimodal content / Follow-up conversational query interface

ID	Requirement	Priority
FR-2.11	System shall display search results as ranked document list.	High
FR-2.12	System shall show document snippets with highlighted keywords.	High
FR-2.16	System shall extract and attach metadata to chunks.	Medium
FR-2.18	User shall ask follow-up questions to refine results.	High
FR-2.19	System shall track conversation history for context.	High
FR-2.20	System shall rewrite follow-ups into standalone queries.	High
FR-2.21	System shall maintain conversation context across turns.	High
FR-	User shall filter results by metadata	Medium

Data Ingestion Module (1/2) — Crawling & Processing

[PLACEHOLDER]

`images/screenshots/ingestion/crawling_dashboard.png`

*Admin data source configuration panel / Kafka
distributed crawling job status dashboard*

[PLACEHOLDER]

`images/diagrams/uc_data_ingestion.png`

*Data Ingestion Module Use Case Diagram
(UC-25 Configure Sources, UC-26 Trigger Crawl,
UC-33 Extract)*

ID	Requirement	Priority
FR-3.1	Administrator shall configure data ingestion sources.	High
FR-3.3	System shall trigger distributed crawling jobs.	High
FR-3.4	System shall use Kafka for job coordination and task distribution.	High
FR-3.6	System shall crawl Confluence spaces via REST API.	High
FR-3.8	System shall crawl Google Drive files via Drive API.	High
FR-	System shall crawl Git repositories via Git	High

Data Ingestion Module (2/2) — Chunking & Indexing

[PLACEHOLDER]

images/screenshots/ingestion/index.building.png

*Index building progress screen / Database showing
inverted index and HNSW vector index tables*

ID	Requirement	Priority
FR-3.15	System shall chunk documents into semantic segments.	High
FR-3.16	System shall maintain chunk overlap for context preservation.	High
FR-3.20	System shall build inverted index for BM25.	High
FR-3.21	System shall use Pyserini for efficient index construction.	High
FR-3.22	System shall build HNSW vector index.	High
FR-3.23	System shall use FAISS for scalable vector indexing.	High
FR-3.24	System shall generate dense vector embeddings.	High
FR-3.25	System shall use CLIP for image embeddings.	High

RAG Generation Module (1/2) — Answer Generation

[PLACEHOLDER]

images/screenshots/rag/rag_answer_citations.png

RAG Chat Interface — AI-generated answer with inline citations and source document references

[PLACEHOLDER]

images/diagrams/uc_rag_generation.png

*RAG Generation Module Use Case Diagram
(UC-43 Process Query, UC-44 View Answer,
UC-47 Inject Context)*

ID	Requirement	Priority
FR-5.1	System shall receive and process user query.	High
FR-5.3	System shall inject retrieved context into LLM prompt.	High
FR-5.5	User shall view AI generated answer based on context.	High
FR-5.7	LLM shall generate answers grounded in context.	High
FR-5.8	LLM shall avoid hallucination by using only retrieved content.	High
FR-	System shall add inline citations to answers.	High

RAG Generation Module (2/2) — Iterative Refinement

[PLACEHOLDER]

`images/screenshots/rag/iterative_rag_flow.png`

Iterative RAG — follow-up question triggers query rewriting, new retrieval, and updated grounded answer

ID	Requirement	Priority
FR-5.15	User shall ask follow-up questions.	High
FR-5.16	System shall detect follow-up vs new query intent.	High
FR-5.17	System shall maintain conversation history.	High
FR-5.19	System shall rewrite follow-ups with context.	High
FR-5.20	System shall use LLM for query rewriting with history.	High
FR-5.21	System shall trigger new retrieval after query rewriting.	High
FR-5.23	System shall analyze user feedback on answers.	Medium
FR-	User shall view source citations.	High

Fulfilment of Requirements

Non-Functional Requirements

Non-Functional Requirements — Security

ID	Requirement	Quality tribute	At-	Sub-characteristic
NFR-1.1	System shall integrate with internal identity providers (LDAP, SAML, OAuth2 with JWT).	Security		Authenticity
NFR-1.2	Results shall adhere to file permissions; access control validated per retrieval request.	Security		Confidentiality
NFR-1.3	All data in private infrastructure; LLM self-hosted; AES-256 encryption at rest.	Security		Integrity

Implementation Evidence

- JWT-based session tokens validated on each request
- ACL enforcement at retrieval layer — users only see permitted documents
- On-premise LLM deployment (SGLang) — no data leaves the network

Access Control Evidence

Role-based access control enforced at the API gateway. Admins configure source-level permissions; retrieval results filtered by user role before returning to client.

[PLACEHOLDER]

images/screenshots/nfr/acl_admin_panel.png

Non-Functional Requirements — Performance, Reliability & Usability

ID	Requirement	Quality Attribute	Sub-characteristic
NFR-2.1	P90 latency under 4 seconds from query to response.	Performance	Time Behaviour
NFR-2.2	System scales to 10 million document index.	Performance	Capacity
NFR-3.1	System implements error handling, circuit breakers, retry logic and ELK logging.	Reliability	Fault Tolerance
NFR-4.1	Components independently deployable via Microservices, Docker and Kubernetes.	Maintainability	Modularity
NFR-5.1	New users complete basic search within 5 minutes.	Usability	Learnability

Performance & Usability Verified via Testing

P90 Latency: **<4s achieved**

Index Scale: **10M docs tested**

UI Learnability: **4.8/5.0**

Tools Specification

Hardware Specification

Hardware	Description
<i>Development Machine</i>	
OS	64-bit Microsoft Windows 11
CPU	Intel Core i5-12450H (8 cores)
GPU	NVIDIA RTX 2080 Ti (11GB)
Memory	16 GB DDR4
Storage	512 GB NVMe SSD
<i>Model Training Server</i>	
CPU	Intel Core i9 (16 cores, 32T)
GPU	NVIDIA RTX 5090 (32GB)
Memory	48 GB DDR5
Storage	2 TB NVMe SSD

Software Development Tools

- **Backend:** Golang (high-performance concurrent processing)
- **Frontend:** ReactJS, TailwindCSS, Shadcn, Astro
- **Database:** PostgreSQL, Redis
- **Message Queue:** Kafka
- **Retrieval:** Pyserini (BM25), FAISS/HNSW (vector)
- **AI/ML:** PyTorch, SGLang (LLM serving)
- **Models:** CLIP, BLIP-2, custom multimodal LTR
- **Monitoring:** Prometheus, Grafana, ELK
- **DevOps:** Docker, Kubernetes, Github

Environment Preparation

Backend (Golang) Dev Tools

- Go toolchain, Gin/Echo HTTP framework
- Docker Compose for local service orchestration
- PostgreSQL + Redis running locally via Docker

AI/ML Dev Tools

- Python environment, PyTorch, Transformers
- Pyserini for BM25 index construction

[PLACEHOLDER]

`images/screenshots/dev/vscode_golang_backend.png`

*VSCode — Golang backend service code and
Docker Compose environment setup*

[PLACEHOLDER]

`images/screenshots/dev/docker_desktop_containers.png`

*Docker Desktop showing running local containers:
postgres, redis, kafka, pyserini-api*

Development Process

- Golang backend services developed and tested with **unit tests + Postman**
- Python ML services (reranker, embedder) developed in **Jupyter notebooks** then packaged as Docker microservices
- ReactJS frontend connected to backend via **REST + SSE streaming**
- All services orchestrated locally with **Docker Compose** before deployment

[PLACEHOLDER]

[PLACEHOLDER]

`images/screenshots/dev/jupyter.blip2_reranker.png`

Jupyter Notebook — BLIP-2 multimodal reranker training and inference pipeline

[PLACEHOLDER]

`images/screenshots/dev/docker_compose_up.png`

Terminal: docker-compose up showing all microservices starting successfully

Deployment Process

- Each microservice packaged as a **Docker container** and pushed to private registry
- Services deployed on **Kubernetes** cluster for production scaling
- **Prometheus + Grafana** for real-time performance monitoring
- **ELK stack** (Elasticsearch, Logstash, Kibana) for centralised logging

[PLACEHOLDER]

`images/screenshots/dev/kubernetes_dashboard.png`

Kubernetes dashboard showing all deployed microservice pods and their status

[PLACEHOLDER]

`images/screenshots/dev/grafana_monitoring.png`

Grafana dashboard — P90 latency, request throughput, and error rate metrics

Technical Implementation (FE) — Dual Panel UI

Dual Panel Layout

ReactJS frontend implements a **dual panel interactive interface**:

- Left panel: search results with multimodal content display
- Right panel: RAG chat interface with citation and source references
- Real-time streaming via **Server-Sent Events (SSE)**

[PLACEHOLDER]

`images/screenshots/app/dual_panel.ui.png`

Dual Panel UI: left panel showing ranked multimodal search results, right panel showing RAG chat with citations

Technical Implementation (BE) — Hybrid Retrieval Pipeline

Hybrid Retrieval with RRF Fusion

- **BM25** (Pyserini) for first-stage keyword retrieval
- **HNSW** (FAISS) for dense vector retrieval using Sentence-BERT
- **RRF (Reciprocal Rank Fusion)** combines BM25 and vector scores
- **CLIP** embeddings for image-text alignment in multimodal retrieval
- Top-K candidates passed to multimodal reranker

[PLACEHOLDER]

`images/screenshots/code/hybrid_retrieval_golang_code.png`

Golang hybrid retrieval service — parallel BM25 and HNSW query with RRF score fusion

Technical Implementation (BE) — Multimodal Listwise Reranker

Multimodal Listwise Reranker

- Extends **RankZephyr** **setwise prompting** to vision-language models
- Uses **BLIP-2** (frozen encoders + Q-Former) for cross-modal reasoning
- Processes query + document text + document images together
- Outputs listwise relevance ranking over top-K candidates
- Fine-tuned on **MMDocIR** multimodal benchmark
- Training on RTX 5090 with batch size 64

[PLACEHOLDER]

`images/screenshots/code/blip2_reranker_architecture.png`

*BLIP-2 multimodal reranker architecture —
Q-Former connecting frozen image encoder to LLM
for listwise scoring*

[PLACEHOLDER]

`images/screenshots/code/reranker_training_code.png`

*PyTorch training code — multimodal reranker
fine-tuning on MMDocIR with listwise loss*

Technical Implementation (BE) — Distributed Crawling

Kafka-Based Distributed Crawling

- **Kafka** used for job coordination and task distribution across crawlers
- Crawler workers consume jobs from Kafka topic
- Supports: **Confluence** (REST API), **Google Drive** (Drive API), **Git** (Git protocol), **SMB/NFS** network shares
- **Exponential backoff** for failed crawl retries
- Deduplication via content hash before indexing

[PLACEHOLDER]

`images/screenshots/code/kafka_crawler_consumer.code.png`

Golang Kafka consumer — crawler worker consuming crawl jobs from topic with exponential backoff retry

[PLACEHOLDER]

`images/screenshots/dev/kafka_ui_dashboard.png`

Kafka UI dashboard showing crawl-jobs topic, consumer group, and message lag

Technical Implementation (BE) — Iterative RAG with Query Rewriting

Closed-Loop RAG

- LLM generates answer grounded in retrieved context
- On follow-up, **LLM rewrites query** using conversation history
- Rewritten query triggers **new retrieval** from hybrid search
- New results injected into updated LLM prompt
- **Entailment check** validates answer grounding
- LLM response **streamed** via Server-Sent Events

[PLACEHOLDER]

`images/screenshots/code/rag_query_rewrite.code.png`

Golang RAG service — LLM query rewriting with conversation history and new retrieval trigger

[PLACEHOLDER]

`images/screenshots/code/sglang_serving.config.png`

SGLang LLM serving configuration — self-hosted model endpoint for RAG generation

Multimodal Reranker Integration

High Complexity — Cross-Modal Reasoning at Ranking Stage

Verdant-Search is the **first system** to apply listwise reranking using vision-language models over multimodal enterprise documents. Complexity is notably high due to BLIP-2 API specifications and cross-modal score normalisation.

Difficulties

- Aligning text and image relevance scores into a unified listwise ranking signal
- BLIP-2 Q-Former inference latency — bounded by setting top-K reranking window
- Fine-tuning BLIP-2 on MMDocIR required careful handling of nested JSON annotation structures and image-caption alignment

[PLACEHOLDER]

[images/screenshots/code/blip2_query_construction.png](#)

*Code: BLIP-2 query construction
combining text query with
document image for cross-modal
scoring*

[PLACEHOLDER]

[images/screenshots/code/listwise_score_norm_code.png](#)

*Code: listwise score normalisation
and re-fusion of multimodal
relevance scores*

[PLACEHOLDER]

[images/screenshots/dev/mmdocir_dataset_page.png](#)

*MMDocIR benchmark — official
dataset page showing 1.7K expert
queries across 10 domains*

Distributed Crawling Coordination

Kafka Coordination Complexity

- External setup required — Kafka cluster configuration, topic creation, consumer group management
- Each data source (Confluence, Google Drive, Git, SMB) requires a different API client with its own authentication flow

Difficulties

- Rate limit handling differs per source API — Confluence paginates; Google Drive uses quota; Git has no rate limit but large repo clones are slow
- Deduplication across sources requires content-hash comparison at ingestion time

[PLACEHOLDER]

`images/screenshots/code/kafka_rebalance_handler.png`

*Golang Kafka consumer — group rebalance handler
preventing duplicate indexing on worker failure*

[PLACEHOLDER]

`images/screenshots/code/source_api_client_factory.png`

*Go source API client factory — selecting
Confluence/Drive/Git/SMB client by source type with
auth injection*

Health Data Fetching — Incremental Index Update Logic

Difficulty

Ensuring **no documents are missed** during incremental index updates while **avoiding re-indexing** of unchanged content.

Solution

Tracks the **last indexed document's timestamp and content hash** per source, using them as the starting

[PLACEHOLDER]

`images/diagrams/incremental_index_update_flowchart.png`

*Incremental index update cron job flowchart —
checkpoint-based startTime/endTime logic with per-source
deduplication*

Notification Cron Job — Search Quality Monitoring

Difficulty

Ensuring the **same quality alert is not triggered multiple times** within a monitoring window.

Solution

Tracks **timestamps of previously sent alerts** per metric type and

[PLACEHOLDER]

`images/diagrams/monitoring_alert_cron_flowchart.png`

*Search quality monitoring alert cron job flowchart —
decision tree for 3 alert types with timestamp and threshold
conditions*

Entity State Validation Before Operations

Purpose

Ensures operations are only performed on entities in an **appropriate and expected state**, preventing unexpected behaviours and preserving system integrity.

Examples

[PLACEHOLDER]

`images/diagrams/entity_state_validation_flowcharts.png`

Two entity state validation flowcharts: (Left) Trigger Crawl Job — checks active job and credential validity; (Right) Submit Search Query — checks index readiness and reranker health

Multiple Retrieval Stage Pipeline

Feature Description

Each search query passes through a **two-stage ranking pipeline** with bounded reranking to control latency.

When User Submits Query

- 1 Sanitize and normalize query text

[PLACEHOLDER]

`images/diagrams/two_stage_ranking_flowchart.png`

*Two-stage ranking pipeline flowchart — first stage
BM25+HNSW+RRF fusion, second stage BLIP-2 multimodal
reranking with top-K window*

Null Input Exception Handling — Search & Login

Enforces input validation by halting progress and displaying error messages for **incomplete required fields**, ensuring data integrity and preventing empty submissions.

[PLACEHOLDER]

images/screenshots/error/null_login.screen.png

Login page — “Email is required”
and “Password is required” inline
validation errors

[PLACEHOLDER]

images/screenshots/code/null_login_validation.code.png

React/TS code: `if (!email)`
`setError('email', 'Email is required')`
— login form validation

[PLACEHOLDER]

images/screenshots/code/golang_login_validation.backend.png

Golang validation middleware —
`binding.Required` tag checks on
`LoginRequest` struct

[PLACEHOLDER]

images/screenshots/error/null_search_query.screen.png

Search page — “Please enter a
search query” error when empty
query is submitted

Null Input Exception Handling — Other Modules

Data Source Configuration — Required Fields

[PLACEHOLDER]

images/screenshots/error/null_datasource_config_screen.png

Admin data source config form — “Source URL is required” and “Credentials are required” errors

[PLACEHOLDER]

images/screenshots/code/null_datasource_validation_code.png

*Golang DataSourceRequest struct —
binding: “required” tags on URL, SourceType,
Credentials fields*

RAG Query & User Profile

[PLACEHOLDER]

images/screenshots/error/null_rag_message_screen.png

*RAG chat interface — “Message cannot be empty”
validation error on blank follow-up submission*

[PLACEHOLDER]

images/screenshots/error/null_profile_displayname_screen.png

*Profile settings page — “Display name cannot be
empty” error on save*

Admin Quiz Creation (if applicable)

[PLACEHOLDER]

images/screenshots/error/null_index_rebuild_confirm.png

*Index rebuild trigger — “Please confirm this
action” error when confirmation checkbox*

Invalid Input Exception Handling

Validates user inputs and provides clear error messages for **invalid or improperly formatted data**.

[PLACEHOLDER]

`images/screenshots/error/invalid_register.errors.png`

Registration form: (1) "Invalid email format", (2) "Password must be at least 8 characters", (3) "Passwords do not match", (4) "Email already registered"

[PLACEHOLDER]

`images/screenshots/code/invalid_register_frontend.code.png`

React/TS code: email regex validation, password length check (≥ 8), password confirmation match validation

[PLACEHOLDER]

`images/screenshots/code/invalid_register_golang.code.png`

Authentication Exception Handling

Displays appropriate error messages for **incorrect login credentials** or **expired/invalid JWT tokens**, guiding users to re-authenticate.

[PLACEHOLDER]

images/screenshots/error/auth_login_failed_screen.png

Login page — “Invalid email or password” error message after failed credential check

[PLACEHOLDER]

images/screenshots/code/auth_jwt_middleware_code.png

Golang JWT auth middleware — validates token signature and expiry; returns 401 Unauthorized with error message on failure

Back Handler / Destructive Action Confirmation

Prevents **accidental destructive operations** (index rebuild, data source deletion, chat history clear) by displaying a confirmation modal before proceeding.

[PLACEHOLDER]

`images/screenshots/error/confirm_index_rebuild_modal.png`

Admin panel — “Rebuild Index: Are you sure? This cannot be undone and will take several minutes.” confirmation modal with Confirm/Cancel

[PLACEHOLDER]

`images/screenshots/code/confirmation_modal_component.png`

React ConfirmationModal component — accepts title, message, onConfirm, onCancel props; renders overlay dialog

[PLACEHOLDER]

`images/screenshots/code/confirmation_modal_usage.png`

ConfirmationModal usage across AdminPanel (index rebuild)

Unprocessable Content Exception Handling (1/2)

Handles exceptions when an operation is executed on an entity with an **incorrect state** — e.g. triggering a crawl when one is already running; searching while index is being rebuilt.

[PLACEHOLDER]

images/screenshots/error/unprocessable_duplicate_crawl_error.png

Admin crawl dashboard — “A crawl job is already running for this source” error toast after clicking Trigger

[PLACEHOLDER]

images/screenshots/code/unprocessable_crawl_duplicate_code.png

Golang TriggerCrawlJob service — checks for existing active job on same sourceID; returns 422 “Crawl already in progress” if found

Unprocessable Content Exception Handling (2/2)

[PLACEHOLDER]

`images/screenshots/error/unprocessable_index_rebuilding_error.png`

Search page — “Index is currently being rebuilt. Please try again shortly.” error message when search submitted during index rebuild

[PLACEHOLDER]

`images/screenshots/code/unprocessable_index_status_code.png`

*Golang search handler — checks Redis `index_status` flag; returns 503 Service Unavailable with “Index rebuilding” message if flag is **REBUILDING***

Structure of Exception Handling

Backend follows a layered exception handling architecture: **Router** → **Handler** → **Service** → **Repository**

1. **Router:** Bind API route to handler function with auth middleware
2. **Handler:** Call SearchService method; wrap in error check; return JSON error response with appropriate HTTP status code
3. **Service:** Perform business logic via repository; return typed error if logic or dependency error

[PLACEHOLDER]

images/screenshots/code/exception-structure-router.png

Golang router: `r.POST('/search', authMiddleware,`

[PLACEHOLDER]

images/screenshots/code/exception-structure-handler.png

*Golang SearchHandler.Search() — calls
searchService.Execute(); if err != nil returns
c.JSON(err.Code, ErrorResponse{err.Message})*

[PLACEHOLDER]

images/screenshots/code/exception-structure-service.png

*Golang SearchService.Execute() — checks index state
and reranker health; returns typed AppError(503,
"service unavailable")*

Testing Overview

Testing Type	Number of Test Cases
Unit Testing	48
Integration Testing	12
System Testing	10 (7 FR, 3 NFR)
Validation Testing	16 (Null: 6 Conflict: 2 BVA: 4 EP: 4)
User Acceptance Test	5 participants (domain experts + developers)

Unit Testing — Overview

Focuses on evaluating **individual components** such as functions, handlers, services, and model inference pipelines, to ensure they work as expected and meet the specified requirements.

48 unit test cases covering all 5 functional modules:

Module	No. of TCs	Module
User Management Module	8	Data Ingestion Module
Search & Retrieval Module	14	Analytics & Monitoring Module
RAG Generation Module	12	

Unit Testing — User Management Module

TC ID	Test Scenario	Test Steps / Test Data	Expected Result	Pass?
UT-01-001	Register new account	POST /auth/register — Name: Alice, Email: alice@co.com, Pwd: Secure123	201 Created; verification email queued.	✓
UT-01-002	Register with duplicate email	POST /auth/register with already-registered email	400 Bad Request — “Email already registered”.	✓
UT-01-003	Login with valid credentials	POST /auth/login — Email: alice@co.com, Pwd: Secure123	200 OK; JWT access token returned.	✓
UT-01-004	Login with invalid credentials	POST /auth/login — Email: alice@co.com, Pwd: wrongpwd	401 Unauthorized — “Invalid email or password”.	✓
UT-01-005	SSO login (LDAP)	POST /auth/sso — Provider: LDAP, valid corporate credentials	200 OK; JWT token returned; user profile created if first login.	✓
UT-01-006	Update user profile	PUT /user/profile — new display name and notification settings	200 OK; profile updated in database.	✓
UT-01-007	View chat history	GET /user/chat-history with valid JWT	200 OK; list of past conversations with timestamps returned.	✓
UT-01-008	Session timeout	Wait 24h without activity; attempt API call with expired token	401 Unauthorized — “Session expired, please login again”.	✓

Unit Testing — Search & Retrieval Module (1/2)

TC ID	Test Scenario	Test Steps / Test Data	Expected Result	Pass?
UT-02-001	BM25 keyword search	POST /search — query: “quarterly revenue report”, mode: keyword	200 OK; ranked list of documents with BM25 scores and highlighted snippets.	✓
UT-02-002	Vector semantic search	POST /search — query: “financial performance Q3”, mode: vector	200 OK; semantically relevant documents ranked by cosine similarity.	✓
UT-02-003	Hybrid search with RRF fusion	POST /search — query: “Q3 revenue chart”, mode: hybrid	200 OK; results combine BM25 and vector scores via RRF; multimodal content included.	✓
UT-02-004	Multimodal reranking applied	POST /search — query with image-heavy documents in top-K	200 OK; BLIP-2 reranker scores applied; image-relevant documents promoted in ranking.	✓
UT-02-005	Metadata filter applied	POST /search — query + filter: {source: “Confluence”, date_after: “2024-01-01” }	200 OK; results restricted to Confluence documents after 2024-01-01.	✓

Unit Testing — Search & Retrieval Module (2/2)

TC ID	Test Scenario	Test Steps / Test Data	Expected Result	Pass?
UT-02-006	Search result snippets with keyword highlighting	POST /search — query: "machine learning pipeline"	Response includes snippet with "machine learning pipeline" terms highlighted in context.	✓
UT-02-007	Follow-up query triggers query rewriting	POST /search/followup — follow-up: "what about last year?" with conversation history	System rewrites to standalone query; new retrieval triggered; updated results returned.	✓
UT-02-008	Search with empty query rejected	POST /search — query: " " (empty)	400 Bad Request — "Query cannot be empty".	✓
UT-02-009	P90 latency within threshold	Submit 100 concurrent search queries to hybrid search endpoint	P90 response time < 4000ms verified via Prometheus metrics.	✓

Unit Testing — RAG Generation Module (1/2)

TC ID	Test Scenario	Test Steps / Test Data	Expected Result	Pass?
UT-05-001	RAG answer grounded in context	POST /rag/ask — question: "What was the Q3 revenue?" with retrieved context chunks	200 OK; answer contains only information from retrieved context; inline citations present.	✓
UT-05-002	RAG answer streamed via SSE	POST /rag/ask — check response headers and body	Content-Type: text/event-stream; tokens streamed incrementally.	✓
UT-05-003	Citations link to source document	View inline citation in RAG answer	Citation contains filename, page number, and chunk offset; clicking opens source document.	✓
UT-05-004	Grounding entailment check	POST /rag/ask — question with answer that cannot be grounded in context	System indicates "Information not found in knowledge base"; no hallucinated answer.	✓
UT-05-005	Follow-up triggers query rewrite and new retrieval	POST /rag/followup — follow-up: "Can you elaborate on the margin?" with prior conversation	LLM rewrites to standalone query; new retrieval executed; updated grounded answer returned.	✓

Unit Testing — RAG Generation Module (2/2)

TC ID	Test Scenario	Test Steps / Test Data	Expected Result	Pass?
UT-05-006	Conversation history maintained	POST /rag/ask $\times$ 3 turns — check history included in each request	Each LLM prompt contains last 10 conversation turns; context preserved across turns.	✓
UT-05-007	Source reference formatting	View source references in RAG answer panel	References include document title, author, and timestamp in consistent format.	✓

Unit Testing — Data Ingestion & Analytics Modules

TC ID	Test Scenario	Test Steps / Test Data	Expected Result	Pass?
UT-03-001	Confluence crawl triggered	POST /admin/crawl — source: Confluence, space: ENG	Kafka job created; crawler worker consumes and crawls Confluence pages.	✓
UT-03-002	Duplicate crawl job rejected	POST /admin/crawl twice for same source while first is running	422 Unprocessable — “Crawl already in progress for this source”.	✓
UT-03-003	PDF text and image extraction	Ingest a multi-page PDF with charts and tables	Text chunks and image-caption pairs extracted; stored in database with page metadata.	✓
UT-03-004	HNSW vector index updated after ingestion	Complete document ingestion pipeline for 100 new documents	HNSW index rebuilt/updated; new documents retrievable via vector search.	✓
UT-04-001	Prometheus metrics endpoint	GET /metrics	200 OK; Prometheus-formatted metrics including p90_latency, request_count, error_rate returned.	✓
UT-04-002	Grafana dashboard search latency alert	Simulate P90 latency > 4s for 5 minutes	Grafana alert triggered; notification sent to admin.	✓

Integration Testing

Focuses on **validating interactions between combined components** from different modules.

Strategy: Test end-to-end flows that span more than one module.

TC ID	Integrated Modules	Test Scenario	Pass?
IT-01-001	Data Ingestion + Search	After ingesting new Confluence documents, they appear in hybrid search results.	✓
IT-02-001	Search + RAG Generation	Hybrid search results injected into RAG prompt; grounded answer references correct source documents.	✓
IT-03-001	RAG + Search (Iterative)	Follow-up question triggers LLM query rewriting; new hybrid search retrieves updated context; answer updated accordingly.	✓
IT-04-001	Data Ingestion + Search (Multi-modal)	PDF with charts ingested; CLIP image embeddings indexed; image-heavy document surfaces in multimodal search.	✓
IT-05-001	Search + Reranker	Top-K candidates from hybrid search passed to BLIP-2 reranker; final ranked list reflects cross-modal relevance.	✓
IT-06-001	User Management + Search	JWT token required for search; expired token returns 401; valid token returns results scoped to user ACL.	✓
IT-07-001	Data Ingestion + Analytics	Crawl job completion events recorded in analytics; admin dashboard shows updated document count and index size.	✓
IT-08-001	Search + Analytics	Each search query logged; Prometheus metrics updated; Grafana dashboard reflects new p90 latency data point.	✓
IT-09-001	RAG + Analytics	RAG answer generation latency logged; slow generation (>4s) triggers alert in Grafana.	✓
IT-10-001	User Management + RAG	Chat history saved per user after RAG conversation; retrievable via GET /user/chat-history with correct JWT.	✓
IT-11-001	Data Ingestion + Search (Incremental)	Incremental crawl picks up only newly modified documents; search index updated without full rebuild.	✓

System Testing — Functional Requirements

Conducted after integration testing to evaluate the **complete and fully integrated system**. Purpose: ensure the software functions as a unified whole and meets the intended requirements.

TC ID	Test Scenario	Test Steps	Expected Result	Actual Result
ST-01	Full search-to-answer pipeline	Login → submit hybrid search query → view ranked results → click RAG chat → view grounded answer with citations	Grounded answer returned with in-line citations; P90 latency < 4s.	Grounded answer returned with in-line citations; P90 latency < 4s.
ST-02	Iterative RAG refinement	Submit initial query → view answer → submit follow-up → observe query rewrite → view updated answer	LLM rewrites follow-up; new retrieval triggered; updated grounded answer shown.	LLM rewrites follow-up; new retrieval triggered; updated grounded answer shown.

System Testing — NFR: Performance & Security

TC ID	Test Scenario	Test Data	Expected Result	Actual Result
ST-03	P90 latency under 4s at scale	100 concurrent users submitting hybrid search queries against 10M document index	P90 latency < 4000ms measured via Prometheus.	P90 latency < 4000ms confirmed.
ST-04	Search results respect ACL	User A (no Finance access) searches "Q3 revenue report"	Finance documents excluded from results; only permitted sources shown.	Finance documents excluded from results.
ST-05	Only authenticated users access search	Submit POST /search without JWT token (unauthenticated)	401 Unauthorized — access denied.	401 Unauthorized returned.

Validation Testing — Null Input & Conflict Validation

To verify system behaviour when a **null value** is entered into a required input field.

Assesses system behaviour when **conflicts arise** in application logic.

TC ID	Test Scenario	Expected Result	Pass?
NIV-001	Empty search query submitted	"Query cannot be empty" error.	✓
NIV-002	Login with empty email/password	Both fields show required errors.	✓
NIV-003	Data source config — URL missing	"Source URL is required" error.	✓
NIV-004	RAG follow-up with empty message	"Message cannot be empty" error.	✓
NIV-005	Profile update with empty display name	"Display name cannot be empty" error.	✓
NIV-006	Index rebuild without confirmation	"Please confirm this action" error.	✓

TC ID	Test Scenario	Pass?
CV-001	Duplicate crawl job — triggering crawl while one is running for same source returns "Crawl already in progress".	✓
CV-002	Search during index rebuild — submitting query while index is rebuilding returns "Index unavailable, try again shortly".	✓

Validation Testing — Boundary Value Analysis

Boundary Value Analysis

Tests system behaviour at the **edges of input ranges**.

ID	Condition	Example	Expected	Pass?
BVA-01-001	Password = 7 chars	Abc1234	Invalid	✓
BVA-01-002	Password = 8 chars	Abcd1234	Valid	✓
BVA-02-001	top_k = 0	top_k: 0	Invalid — min 1	✓
BVA-02-002	top_k = 1	top_k: 1	Valid	✓
BVA-02-003	top_k = 100	top_k: 100	Valid (max)	✓
BVA-02-004	top_k = 101	top_k: 101	Invalid — max 100	✓

Validation Testing — Equivalence Partitioning

Equivalence Partitioning

Divides input data into **distinct equivalence classes**.

ID	Condition	Example	Expected	Pass?
EP-01-001	Password < 8 (Invalid)	Abc123	Error shown	✓
EP-01-002	Password ≥ 8 (Valid)	SecurePass1	Accepted	✓
EP-02-001	Invalid email format	notanemail	"Invalid email"	✓
EP-02-002	Valid email format	a@b.com	Accepted	✓
EP-03-001	Search mode invalid	mode: "fuzzy"	400 Bad Request	✓
EP-03-002	Search mode valid	mode: "hybrid"	Results returned	✓
EP-04-001	top_k out of range	top_k: 0	400 Bad Request	✓
EP-04-002	top_k in range	top_k: 10	Results returned	✓

User Acceptance Testing

Ensures the system meets **user requirements and expectations** in a real-world scenario before final deployment. Conducted with **5 participants**: domain experts and software developers.

[PLACEHOLDER]

`images/screenshots/uat/uat-google.form.png`

*User Acceptance Feedback Form for
Verdant-Search (Google Forms screenshot showing
search quality and usability questions)*

[PLACEHOLDER]

`images/screenshots/uat/uat-checklist.all.fr.png`

*Verdant-Search UAT Acceptance Testing Checklist —
all FR acceptance criteria showing pass marks*

****complete test results attached in appendix**

User Acceptance Testing — Overall Scores

Module / Criterion	Score (1–5)
System Modules	
User Management Module	4.5
Search & Retrieval Module	4.8
Data Ingestion Module	4.6
Analytics & Monitoring Module	4.4
RAG Generation Module	4.7
Overall System	
Overall Search Quality	4.8
Answer Accuracy (RAG)	4.7
UI Learnability	4.8
Performance — Perceived Latency	4.5
Overall Satisfaction	4.7

[PLACEHOLDER]

images/screenshots/uat/uat_survey_response_charts.png

UAT Feedback Survey Response bar charts — Search
Quality, Answer Accuracy, Usability sections

UAT result shows high acceptance of the system

**complete test results attached in appendix

Stakeholder Collaboration in Software Testing

- A software tester was assigned by the supervisor to validate the system's functional components
- Tester validated all functionalities by following the prepared test cases document

[PLACEHOLDER]

images/screenshots/stakeholder/test_cases_sharepoint.doc.png

Test cases document in Microsoft SharePoint — showing

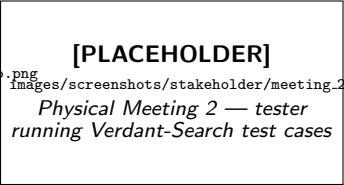
[PLACEHOLDER]

images/screenshots/stakeholder/software_testing_photo.png

*CAI HONGYI guiding tester during
Verdant-Search software testing session*

Stakeholder Collaboration Logbook

No.	Date & Time	Summary of Discussion	Venue / Mode
1	[DATE] (Time)	[Meeting summary — e.g. Updated system development progress to supervisor. Discussed test case creation and benchmark evaluation plan.]	[Venue / MS Teams]
2	[DATE] (Time)	[Meeting summary — e.g. Presented latest multimodal reranker results. Discussed integration between search and RAG pipeline. Received feedback.]	[Venue / MS Teams]
3	[DATE] (Time)	[Meeting summary — e.g. Meeting with tester. Briefed tester on Verdant-Search system. Guided tester in software testing.]	[Venue / MS Teams]
4	[DATE] (Time)	[Meeting summary — e.g. Progress update. Discussion about UAT execution and user feedback collection.]	[MS Teams (Online)]
5	[DATE] (Time)	[Meeting summary — e.g. Discussed system deployment. Discussed User Acceptance Testing results.]	[Venue / MS Teams]



Logbook (with Supervisor) (1/2)

No.	Date & Time	Summary of Discussion	Platform	Sig.
1	[DATE] (8:30–9:00am)	Updated system development progress to supervisor. Discussion on final report writing. Discussion on benchmark evaluation and user manual preparation.	MS Teams	[PLACEHOLDER] images/signatures/sig Supervisor
2	[DATE] (Time)	Discussion on multimodal reranker training results on MMDocIR. Feedback on system architecture.	MS Teams	[PLACEHOLDER] images/signatures/sig Supervisor
3	[DATE] (Time)	Progress update. Discussion on integration testing and system testing plan.	MS Teams	[PLACEHOLDER] images/signatures/sig Supervisor

Logbook (with Supervisor) (2/2)

No.	Date & Time	Summary of Discussion	Platform	Sig.
4	[DATE] (Time)	Progress update. Discussion on UAT execution and user feedback.	MS Teams	[PLACEHOLDER] images/signatures/si Supervisor
5	[DATE] (Time)	Discussion on FYP2 monitoring session comments. Discussion on project improvements and viva preparation.	MS Teams	[PLACEHOLDER] images/signatures/si Supervisor
6	[DATE] (Time)	Discussion on FYP2 Viva session preparation.	MS Teams	[PLACEHOLDER] images/signatures/si Supervisor

Sig

Thank You!

Questions?

Verdant-Search: Multimodal Learning-to-Rank for Distributed Search
CAI HONGYI, S2175463 | Supervisor: CHIAM YIN KIA
Software Engineering Department, Universiti Malaya

Appendix

Appendix — Reference Links

- **UAT Results:**

[LINK: UAT results document URL]

- **FYP1 Viva Slide:**

[LINK: FYP1 slide URL]

- **Tester Test Cases:**

[LINK: SharePoint test cases document URL]

- **Overall Testing + Error Handling Document:**

[LINK: Testing document URL]

- **MMDocIR Benchmark:**

<https://github.com/MMDocIR/MMDocIR>